```
# K-NEAREST NEIGHBORS (KNN) - DIABETES PREDICTION
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score
)
# 2. LOAD DATASET
from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')
# Read dataset
df = pd.read_csv('/content/drive/MyDrive/diabetes.csv')
# 3. READ THE DATASET

print("First 5 rows:")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nDataset information:")
print(df.info())

# ============================================================
# 4. CHECK FOR MISSING VALUES
```

```
# ============================================================

print("\nMissing values:")
print(df.isnull().sum())

# ============================================================
# 5. SEPARATE FEATURES AND TARGET
# ============================================================

# X = independent/input features
# y = dependent/target variable

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print("Outcome")

# ============================================================
# 6. DIVIDE DATA INTO TRAINING AND TESTING SETS
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining data shape:")
```

```
print(X_train.shape)

print("\nTesting data shape:")
print(X_test.shape)

# ============================================================
# 7. NORMALIZE / STANDARDIZE THE FEATURES
# ============================================================

scaler = StandardScaler()

# Fit scaler on training data
X_train = scaler.fit_transform(X_train)

# Apply same transformation to test data
X_test = scaler.transform(X_test)

# ============================================================
# 8. CREATE KNN MODEL
# ============================================================

knn = KNeighborsClassifier(
    n_neighbors=5
)

# ============================================================
# 9. TRAIN THE MODEL
# ============================================================

knn.fit(X_train, y_train)

# ============================================================
# 10. MAKE PREDICTIONS
# ============================================================
```

```
y_pred = knn.predict(X_test)

# ============================================================
# 11. CONFUSION MATRIX
# ============================================================

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display confusion matrix graphically

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Diabetes", "Diabetes"],
    yticklabels=["No Diabetes", "Diabetes"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")

plt.show()

# ============================================================
# 12. ACCURACY
# ============================================================
```

```
accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100)

# ============================================================
# 13. ERROR RATE
# ============================================================

error_rate = 1 - accuracy

print("\nError Rate:")
print(error_rate)

print("\nError Rate Percentage:")
print(error_rate * 100)

# ============================================================
# 14. PRECISION
# ============================================================

precision = precision_score(
    y_test,
    y_pred
)

print("\nPrecision:")
```

```
print(precision)

# ============================================================
# 15. RECALL
# ============================================================

recall = recall_score(
    y_test,
    y_pred
)

print("\nRecall:")
print(recall)

# ============================================================
# 16. PRINT ALL RESULTS
# ============================================================

print("\n======================================")
print("KNN PERFORMANCE")
print("======================================")

print("Accuracy  :", accuracy)
print("Error Rate:", error_rate)
print("Precision :", precision)
print("Recall    :", recall)
```